



UBenchAI-Framework Report

Unified Benchmarking Framework for AI Factory Workloads
on MeluXina

Arianna Amadini, Dennys Huber, Elizabeth Koleva, Alberto Taddei

February 2026



Abstract. This report documents the work delivered during the *Benchmarking AI Factories on MeluXina Supercomputer* challenge, whose outcome is **UBenchAI**, a unified benchmarking framework for AI Factory workloads on the *MeluXina* supercomputer.

The framework provides a repeatable workflow to deploy AI services on a SLURM-based HPC system, run stress tests, collect both client-side and service-side metrics via Prometheus/Grafana, and archive artifacts for reproducible analysis.

We validated the workflow on **vLLM**, **Ollama**, and **Qdrant**, and evaluated results through analysis and scalability trends.

1 Introduction

This report presents the work delivered in the EUMaster4HPC Student Challenge 2025–2026 on benchmarking AI Factory workloads on the MeluXina supercomputer. The main outcome of the project is **UBenchAI**, a unified benchmarking framework designed to evaluate heterogeneous AI services through a common, reproducible workflow.

The challenge is representative of a realistic AI Factory setting. In this context, the quality of a service is not described only by model accuracy or functionality, but also by its ability to:

- handle increasing request load,
- maintain acceptable latency under stress,
- sustain throughput over time,
- expose enough telemetry to understand bottlenecks.

For this reason, we approached benchmarking as a *methodology problem*, not only as a performance measurement task. In our project, a useful benchmarking workflow had to be:

- **Repeatable:** the same configuration should produce comparable runs;
- **Traceable:** parameters, logs, and outputs should be archived for review;
- **Comprehensive:** both client-side and service-side behavior should be visible;
- **HPC-compatible:** deployment and execution should fit SLURM-based cluster usage.

1.1 Scope and services

UBenchAI targets multiple classes of AI Factory workloads through the same CLI and recipe-based execution model. In our implementation, we covered:

- **vLLM:** an inference service exposing native Prometheus metrics;
- **Ollama:** an inference service whose metrics are collected via an exporter;
- **Qdrant:** a vector database service supporting retrieval workloads.

By including both inference and retrieval services, we aimed to show that the framework generalizes across different service types and metric families while keeping a unified experimental procedure.

1.2 How we evaluate results

We evaluate the project on two complementary axes:

1. **Framework-level success:** whether UBenchAI provides a robust and unified workflow to deploy, monitor, stress-test, and archive experiments on MeluXina.
2. **Service-level performance:** observed throughput, latency, and error behavior as load increases, using both client-side measurements and service dashboards.

In practice, our analysis follows three ideas that were central throughout the challenge:

- **Steady-state analysis:** measurements are taken from stable windows after ramp-up;
- **Scalability behavior:** we study how throughput and latency evolve with concurrency;
- **Saturation identification:** we detect when more load stops improving throughput and starts degrading

latency or reliability.

The goal is therefore not a simplistic head-to-head comparison across very different services, but rather a **capacity and saturation profiling workflow** that can be reproduced and extended.

2 UBenchAI Framework Overview

UBenchAI is organized as a workflow-controlled suite of components executed on a SLURM-based HPC environment, with a single command-line interface and recipe-driven configuration. The key design principle is that experiments should follow the same pipeline independently of the service under test.

The repository structure reflects this modular approach (source code under `src/ubenchai`, recipes, dashboards, exporter, tests, templates, results/examples), and the README documents both the CLI usage and a complete monitoring workflow on MeluXina.

2.1 Architecture

The framework is composed of four main components:

1. **Server:** launches and manages the target service (e.g., vLLM, Ollama, Qdrant) from a recipe on SLURM.
2. **Client:** generates stress-test workloads and records request-level outcomes (throughput, latency percentiles, errors).
3. **Monitor:** runs Prometheus and Grafana to collect and visualize service-side metrics.
4. **Logging / Artifacts:** stores configurations, logs, and dashboard exports in a run folder for later comparison.

This separation gives each component a clear responsibility while making integration predictable. It also helps team organization, since the CLI-driven workflow requires these parts to interoperate reliably.

2.2 Unified workflow on MeluXina

Each experiment follows the same sequence:

1. Select the service, monitor, and client recipes.
2. Start the service (`ubenchai server start -recipe <service>`).
3. Start monitoring (`ubenchai monitor start -recipe <mon> -targets <jobid>`).
4. (Ollama only) Start the exporter inside the same job allocation.
5. Open SSH tunnels to access Prometheus/Grafana locally.
6. Inspect dashboards and verify that the run reaches a stable regime.
7. Run client load (`ubenchai client run -recipe <load>`).
8. Stop jobs and save logs/exports in the run folder.

The core benefit of this workflow is consistency. The sequence of actions, monitoring stack, and artifact structure remain stable across services, which makes experiments easier to compare and rerun.

Service	Server recipe	Monitor recipe	Client recipe
vLLM	<code>vllm</code>	<code>vllm-monitor</code>	<code>vllm-stress-test2</code>
Ollama	<code>ollama-tiny</code>	<code>ollama-monitor</code>	<code>ollama-stress-test</code>
Qdrant	<code>qdrant-vectoradb</code>	<code>qdrant-monitor</code>	<code>qdrant-stress-test</code>

Table 1: Representative recipe combinations used for end-to-end experiments.

2.3 Metrics and measurement strategy

UBenchAI combines **client-side** and **service-side** metrics. Client-side metrics capture what a user experiences; service-side metrics provide a closer view of internal behavior and resource usage.

Service	Service-side metrics	Client-side metrics
vLLM	Total tokens/s (prompt + generation), request counters, cache-related indicators	req/s, latency p95, errors
Ollama	GPU utilization, tokens/s (via exporter), service health metrics	req/s, latency p95, success rate
Qdrant	QPS, latency p95, service-side query metrics	QPS, latency p95, errors

Table 2: Metric families collected per service, combining service-side and client-side views.

Across services, throughput captures capacity, latency percentiles capture tail behavior, and errors/failures capture stability under stress. Service-side telemetry is then used to interpret whether bottlenecks come from

hardware saturation, batching/scheduling, or service-specific limits.

3 Experimental Setup and Result Evaluation

This section describes how we interpret the experimental outcomes produced by UBenchAI, following the steady-state and saturation-oriented approach used during the challenge.

3.1 Steady-state windows and ramp-up

A central methodological point is the use of **steady-state stress-test windows**. Stress tests generally include an initial ramp-up phase in which load grows and the system transitions from idle to a loaded regime. During this transient phase, metrics may fluctuate because of warm-up effects, caching, or service initialization.

For reproducibility and interpretability, we focus on time intervals where:

- throughput becomes approximately stable,
- latency distributions do not drift substantially,
- monitoring indicates a consistent operating regime.

This is one of the reasons why Grafana dashboards were important in the project: they were not used only for visualization, but also to validate whether the selected measurement window was meaningful.



Figure 1: Example Grafana dashboard (vLLM) used during benchmarking. After the initial step-up, throughput reaches a stable plateau; this region is used as the steady-state window for extracting representative values.

3.2 Scalability and saturation

We interpret scalability as the regime in which increasing concurrency yields increasing throughput while latency remains acceptable. Saturation is the transition point after which additional load produces diminishing throughput gains and significantly degrades latency or reliability.

In practice, saturation manifests differently across services:

- inference services may saturate due to GPU compute limits, memory bandwidth, batching constraints, or scheduling overheads;
- retrieval services may saturate due to CPU usage, memory pressure, network overhead, or index/collection configuration.

UBenchAI supports this analysis by combining client-side request statistics with service dashboards, allowing us to correlate performance changes with service-side metrics rather than relying on client throughput alone.

3.3 Observed results

Table 3 summarizes representative client-side results extracted from selected steady-state scenarios.

Scenario	Load	Throughput	Latency p95
vLLM (Mistral-7B)	128 users	96.0 req/s	0.115 s
Ollama (TinyLlama)	15 users	3.61 req/s	3.95 s
Qdrant (search)	256 users	353 QPS	9.50 ms

Table 3: Example client-side results from steady-state stress-test windows.

These values should be read as **service-specific profiles under chosen loads**, not as direct service comparisons. The tested services differ in purpose (inference vs retrieval), request semantics, and resource behavior.

What remains directly comparable is the *method*: same deploy-monitor-load-stop workflow, same monitoring logic, and consistent artifact logging.

3.4 What the results tell us

From the experiments performed during the challenge, three takeaways were consistent:

- **Steady-state extraction is useful**: focusing on stable windows yields interpretable throughput and latency values.
- **Scalability holds up to a service-specific limit**: throughput generally improves with concurrency before flattening.
- **Saturation behavior differs across services**: overload appears at different concurrency levels and with different signatures (latency growth, throughput plateau, failures).

A key outcome of the project is therefore not only a set of performance values, but the ability to **reconstruct and review the full experiment**: recipes, SLURM handles, dashboards, logs, and archived outputs.

4 Work Organization and Team Collaboration

This section describes how we organized the work as a team, how tasks were distributed, and which coordination practices helped us deliver a working framework.

4.1 Team structure and task allocation

The project naturally decomposed into parallel development threads aligned with the UBenchAI components. We organized work around four main task families:

- **Service deployment (Server)**: SLURM job scripts, service recipes, and lifecycle management.
- **Load generation (Client)**: workload recipes, concurrency patterns, request logging, and summary statistics extraction.
- **Monitoring (Monitor)**: Prometheus/Grafana integration, exporters (especially for Ollama), and dashboard validation.
- **Experiment management / Logging**: run folder organization, exports, log collection, and documentation support.

In practice, this also aligned with module leads:

- **Dennys Huber**: Server module lead;
- **Alberto Taddei**: Client module lead;
- **Arianna Amadini**: Monitor module lead;
- **Elizabeth Koleva**: Reporting module lead.

This division allowed parallel work while preserving regular integration checkpoints, which were essential because the components had to work together through a shared CLI workflow.

4.2 Work calendar and milestones

We followed an iterative approach with short cycles:

1. **Initial alignment**: define project goals, choose supported services, and agree on evaluation criteria (steady-state, scalability, saturation).
2. **Prototype phase**: run one service end-to-end with monitoring and a minimal stress test.
3. **Generalization**: extend recipes and validate that the workflow remains consistent across services.
4. **Results phase**: run controlled experiments, verify steady-state windows, and consolidate artifacts.
5. **Reporting phase**: synthesize methodology, results, and lessons learned into poster and report material.

This approach reduced integration risk. Instead of building components in isolation for too long, we repeatedly validated the complete pipeline on the cluster.

4.3 Coordination practices

Working on a shared HPC system introduces constraints that strongly affect collaboration: queue times, resource availability, job targeting, and tunnel configuration. To manage these constraints, we relied on:

- **Shared experiment conventions**: consistent naming for runs, recipes, and output folders.
- **Integration checkpoints**: short sessions focused on end-to-end execution and troubleshooting.

- **Clear ownership with peer review:** each component had a primary maintainer, but integration decisions were reviewed together.

A particularly important point was the monitoring pipeline. Prometheus targets, Grafana dashboards, and SSH tunneling required synchronized setup; treating monitoring as a first-class component (rather than a final add-on) improved both the reliability and interpretability of benchmark runs.

5 What We Gained from the Challenge and Group Work

Beyond the technical deliverable, the challenge gave us practical experience in designing reliable experiments on shared HPC infrastructure and collaborating on a multi-component engineering project.

5.1 Technical skills and insights

The main technical gains include:

- **HPC-native deployment:** translating AI service deployment into SLURM-based execution and lifecycle management;
- **Benchmark methodology:** understanding how steady-state analysis, scalability, and saturation complement each other;
- **Monitoring literacy:** combining client-side metrics with service-side dashboards to identify bottlenecks;
- **Reproducibility engineering:** organizing artifacts and logs so results can be reviewed and compared across runs.

We also learned that performance analysis is best understood as a combination of views: a throughput number without context can be misleading, while dashboards without controlled load are difficult to interpret. UBenchAI helps connect these perspectives in a single workflow.

5.2 Teamwork and process lessons

From working in a group, we gained practical lessons that were especially relevant in a constrained HPC environment:

- **Interface-first development:** agreeing early on CLI commands and recipe formats reduced integration friction.
- **Communication under constraints:** scheduling runs and coordinating resources required concise updates and clear responsibility boundaries.
- **Iterative integration:** frequent end-to-end tests prevented late surprises and improved reliability.

The group setting also made the project more robust. Different team members focused on different classes of issues (deployment, workload generation, monitoring, reporting), which helped us catch missing metrics, fragile assumptions, and workflow edge cases earlier.

6 Conclusion

UBenchAI delivers a unified, recipe-driven benchmarking framework for AI Factory workloads on MeluXina. Through a single CLI and a consistent workflow, it deploys services on SLURM, attaches monitoring, generates stress-test load, and archives run artifacts for later analysis.

The framework supports practical performance evaluation by combining client-side metrics (throughput, latency percentiles, errors) with service-side dashboards, enabling steady-state analysis, scalability profiling, and saturation detection.

The challenge showed that reliable benchmarking is as much about **method** as about **numbers**. By focusing on repeatability, traceability, and cross-service consistency, UBenchAI provides a strong foundation for future benchmarking and capacity analysis on HPC-hosted AI services.

Acknowledgements

We thank Dr. Farouk Mansouri for his guidance, feedback, and support throughout the project, and the EUMaster4HPC programme for the organization and access to resources.